our shielding · Your smart contracts, our shielding · Your smart c

shieldify

HLOS

SECURITY REVIEW

Date: 10 March 2026

CONTENTS

1. **About Shieldify Security** 3
2. **Disclaimer** 3
3. **About HLOS** 3
4. **Risk classification** 7
   - 4.1 Impact 7
   - 4.2 Likelihood 7
5. **Security Review Summary** 7
   - 5.1 Protocol Summary 7
   - 5.2 Scope 7
6. **Findings Summary** 9
7. **Findings** 9

1. About Shieldify

Positioned as the first hybrid Web3 Security company, Shieldify shakes things up with a unique subscription-based auditing model that entitles the customer to unlimited audits within its duration, as well as top-notch service quality thanks to a disruptive 6-layered security approach. The company works with very well-established researchers in the space and has secured multiple millions in TVL across protocols, also can audit codebases written in Solidity, Rust, Go, Vyper, Move and Cairo.

Learn more about us at shieldify.org.

2. Disclaimer

This security review does not guarantee bulletproof protection against a hack or exploit. Smart contracts are a novel technological feat with many known and unknown risks. The protocol, which this report is intended for, indemnifies Shieldify Security against any responsibility for any misbehavior, bugs, or exploits affecting the audited code during any part of the project's life cycle. It is also pivotal to acknowledge that modifications made to the audited code, including fixes for the issues described in this report, may introduce new problems and necessitate additional auditing.

3. About HLOS

The Operating System is achieving aggregation between trade and date. This system is built on top of the HyperEVM and all trading is routed through the Hyperliquid Exchange. EVM activity will be native to the chain, including token swaps, adding to LPs, buy/sell NFTs, staking etc. Alpha and data integration will be done through various sources.

```
HyperEVM configs:
  - Public RPC for now: https://rpc.hyperliquid-testnet.xyz/evm
  - Chain ID: 998

For EVM Endpoints calling corewriter, the flow is as follows:
  1. FE asks BE for an action payload (e.g. encode_add_api_wallet)
  2. FE submits to CoreWriter from the 'USERs EOA. (coreWriter.
     sendRawAction(data_from_backend))
  3. FE notifies BE with the tx hash (receive_tx_hash endpoint)
  4. BE stores data from tx, verifies state has changed on hypercore
  "For bridging, the data received is instead used to do an ERC20
     Transfer"
  (https://hyperliquid.gitbook.io/hyperliquid-docs/for-developers/
     hyperevm/hypercore-less-than-gre ater-than-hyperevm-transfers)
  "For all endpoints except compute_bridge_address, in the request
     headers"
  include this key value pair
```

Trading Endpoints:

```
/perps:
/perps/market_order: opens a market order.
  Method: POST Params:
  name: str - required, e.g. ETH
  is_buy: bool - True = long, False = short - required sz: float: amount
     to long/short - required e.g 0.01 slippage: float: optional
  Optional:
    leverage: int            # e.g. 5, 10, 20...
    is_cross: bool           # 'true'/'false' (default true)"

/perps/close_market_order: market closes a position for an asset
  Method: POST Params:
  name: str - name of coin - required, e.g. ETH
  sz: float: amount to close - optional, e.g. 0.01

/perps/limit_order: sets a limit order for an   asset
  Method: POST Params:
  name: str -name of coin- required, e.g. ETH
  sz: float: size of order - required, e.g. 0.01
  px: float: limit price of order - required, e.g. 2200 is_buy: bool -
     True = long, False = short - required slippage: float: optional
```

POST expects:

```
  name: str          # e.g. ""ATOM"" is_buy: bool      # ""true""/""false"
     " px: float          # limit price
  sz: float          # order size Optional:
  leverage: int      # e.g. 5, 10, 20... (perps) is_cross: bool    # ""
     true""/""false"" (default true)
  tif: str           # ""Gtc"" | ""Ioc"" | ""Alo"" (default ""Gtc"")
     reduce_only: bool # default false
```

```
/perps/cancel_order: cancels an existing order:
  name: str -name of coin- required, e.g ETH
  oid: int: order id - required, e.g. 101607794562

/perps/adjust_leverage: adjusts the leverage of an existing position
  POST body expects
    - coin: str
    - newLevAmount: int
    - isCrossMargin: bool"

/perps/modify_order: modifies an existing limit order:
  name: str -name of coin- required, e.g ETH
  oid: int: order id - required, e.g. 101607794562 sz: float: size of
     order - required, e.g. 0.01
  px: float: limit price of order - required, e.g. 2200 is_buy: bool -
     True = long, False = short - required
  order_type: - refer to hyperliquid order types-optional - defaults to:
     "{limit: "{tif: ""Gtc}} (limit order)
  reduce_only: - whether the order only reduces a position - optional -
     defaults to False

/spot:
  https://hyperliquid.gitbook.io/hyperliquid-docs/for-
  developers/api/asset-ids

/spot/limit_order: sets a limit order for a spot asset
  Method: POST Params:
  name: str - pair of coin- required, e.g. PURR/USDC sz: float: size of
     order - required
  px: float: limit price of order - required
  is_buy: bool - True = long, False = short - required slippage: float:
     optional
```

```
/spot/market_order: market order for spot asset
  Method: POST Params:
  name: str - required, e.g. PURR sz: float: size of order - required
  px: float: limit price of order - required
  slippage: float: optional
  "UNIT_ASSETS = {
    ""ETH"":  ""UETH"",
    ""BTC"":  ""UBTC"",
    ""SOL"":  ""USOL"",
    ""FARTCOIN"": ""UFART"",
  }"

/spot/close_market_order:
/spot/cancel_order: cancels an existing order:
  name: str -name of coin- required oid: int: order id - required
  https://hyperliquid.gitbook.io/hyperliquid-docs/trading/order-types

/spot/modify_order: modifies an existing limit order:
  name: str -name of coin- required, e.g ETH
  oid: int: order id - required, e.g. 101607794562
  sz: float: size of order - required, e.g. 0.01
  px: float: limit price of order - required, e.g. 2200 is_buy: bool -
     True = long, False = short - required
  order_type: - refer to hyperliquid order types-optional - defaults to:
     "{limit: "{tif: ""Gtc}} (limit order)
  reduce_only: - whether the order only reduces a position - optional -
     defaults to False
```

EVM Endpoints:

```
/evm/add_api_wallet:
  Method: POST
  Generates an API wallet for the user, then returns the bytes to
     register this wallet in Corewriter.

/evm/approve_builder_fee:
  Method: GET
  Returns the bytes to approve our builder fee.

/evm/get_core_address:
  Method: GET
  Params: token_id - token ID on hypercore
  Returns the correct address to bridge tokens from HyperEVM to the core.
      If the token is not linked, return None

/evm/receive_tx_data:
  Method: GET
  Params: tx_hash - tx hash from transaction. Passes tx hash to backend
     for backend to store
```

4. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4.1 Impact

- **High** – results in a significant risk for the protocol's overall well-being. Affects all or most users
- **Medium** – results in a non-critical risk for the protocol affects all or only a subset of users, but is still unacceptable
- **Low** – losses will be limited but bearable – and covers vectors similar to griefing attacks that can be easily repaired

4.2 Likelihood

- **High** – almost certain to happen and highly lucrative for execution by malicious actors
- **Medium** – still relatively likely, although only conditionally possible
- **Low** – requires a unique set of circumstances and poses non-lucrative cost-of-execution to rewards ratio for the actor

5. Security Review Summary

The security review lasted 5 days with a total of 80 hours dedicated by the Shieldify team.

Overall, the code is well-written. The audit report identified three High, four Medium, and two Low severity issues. They are mainly related to the logging of the private key, csrf_exempt on many POST endpoints, weak authentication and missing input validation.

The Hlos team has been highly responsive to the Shieldify research team's inquiries and promptly implemented all recommendations.

5.1 Protocol Summary

Project Name	**Hlos – Hyperliquid**
Repository	backend
Type of Project	Trading API Backend
Security Review Timeline	5 days
Review Commit Hash	56e9b7bfb7d1ecf14a66844f683218422aace350
Fixes Review Commit Hash	1263189b93f37d0adac9ffceff139a216713b209

5.2 Scope

The following smart contracts were in the scope of the security review:

File	nSLOC
hyperevm/admin.py	3
hyperevm/apps.py	6
hyperevm/models.py	26
hyperevm/tests.py	3
hyperevm/urls.py	11
hyperevm/views.py	119
perps/admin.py	3
perps/apps.py	6
perps/models.py	3
perps/tests.py	3
perps/urls.py	11
perps/views.py	284
rePRICE/asgi.py	16
rePRICE/settings.py	140
rePRICE/urls.py	26
rePRICE/wsgi.py	16
spot/admin.py	3
spot/apps.py	6
spot/models.py	3
spot/tests.py	3
spot/urls.py	11
spot/views.py	244
walletauth/admin.py	3
walletauth/apps.py	6
walletauth/models.py	10
walletauth/serializers.py	30
walletauth/tests.py	3
walletauth/urls.py	9
walletauth/utils.py	74
walletauth/views.py	132
Total	1213

6. Findings Summary

The following number of issues have been identified, sorted by their severity:

- **High** issues: **3**
- **Medium** issues: **4**
- **Low** issues: **2**

ID	Title	Severity	Status
[H-01]	Logging of Private Key / Printing Secrets	High	Fixed
[H-02]	`@csrf\_exempt` on Many POST Endpoints	High	Fixed
[H-03]	Weak Authentication: `get\_public\_key()` Accepts Unverified Header	High	Fixed
[M-01]	Debugging Imports and Pdb Usage	Medium	Fixed
[M-02]	Missing Input Validation and Unsafe Numeric Casting	Medium	Fixed
[M-03]	Returning Raw Exceptions in API Responses	Medium	Fixed
[M-04]	Incorrect `update\_leverage` Parameter Order Causes Wrong Leverage/Margin Mode	Medium	Fixed
[L-01]	`slippage` Parameter Retrieved but Never Used	Low	Fixed
[L-02]	No HTTPS / Secure Cookie Enforcement Settings in `settings.py`	Low	Acknowledged

7. Findings

[H-01] Logging of Private Key / Printing Secrets

Severity

High Risk

Description

The code contains `print(exchange.wallet._private_key)` and multiple `print(...)` statements that output sensitive info.

If the service runs in an environment logging stdout/stderr (systemd, container logs, cloud logs), the private key will be written to logs. Anyone with access to logs (or backups of logs) can retrieve the raw private key and take full control of that wallet/account.

Private keys are the ultimate credentials for blockchain wallets, printing them is equivalent to storing them in plain logs. This finding refers to OWASP: A02 – Cryptographic Failures / Sensitive Data Exposure.

Vulnerable Scenario: Vulnerable code (examples found in repo):

```
print(exchange.wallet._private_key)
print(exchange.account_address)
# many other prints of responses like `print(order_result)` or `print(
    response)`
```

Location of Affected Code

File: perps/views.py

File: spot/views.py

Impact

Full compromise of the blockchain account/wallet used by the app. An attacker can sign arbitrary transactions, withdraw funds, and impersonate the service on the chain.

Recommendations

Remove any print statements that reveal keys or secrets before going to production.

Team Response

Fixed.

[H-02] `@csrf_exempt` on Many POST Endpoints

Severity

High Risk

Description

Both `perps/views.py` and spot/views.py import and use `@csrf_exempt` on multiple endpoints.

If the endpoint relies on cookie-based authentication (sessions), an attacker could craft a malicious page that causes a victim's browser to POST to these endpoints and perform actions as the victim (Cross-Site Request Forgery). If token-based auth is not enforced, the CSRF exemption broadens the attack surface.

CSRF protections exist to prevent a third-party website from causing stateful actions in users' browsers. Removing them requires you to enforce alternate robust authentication (e.g., bearer tokens in headers) and to ensure endpoints are safe to call cross-origin; the code does not show such compensating controls.

This refers to OWASP: A05 – Security Misconfiguration / A07 – Identification and Authentication Failures (CSRF and auth bypass risk).

Vulnerable Scenario: Vulnerable code examples: (file shows multiple @csrf_exempt occurrences)

```
from django.views.decorators.csrf import csrf_exempt
@csrf_exempt
@csrf_exempt
@csrf_exempt
```

Location of Affected Code

File: perps/views.py

File: spot/views.py

Impact

Unauthorized actions such as placing or cancelling orders on behalf of authenticated users, changing user data if endpoints permit.

Recommendations

Re-enable CSRF protection for endpoints that are used by browsers. Remove `@csrf_exempt` **unless there is a compelling reason.**

Team Response

Fixed.

[H-03] Weak Authentication: `get_public_key()` Accepts Unverified Header

Severity

High Risk

Description

The `get_public_key()` function extracts the "Authorization: PublicKey" header and treats it as an authenticated identity without verification.

Vulnerable code:

```
def get_public_key(request):
    auth_header = request.headers.get("Authorization")
    if not auth_header or not auth_header.startswith("PublicKey "):
        return None
    return auth_header.split(" ")[1]
```

This refers to OWASP Mapping: A07:2021 – Identification and Authentication Failures, API2:2023 – Broken Authentication

Vulnerable Scenario: Send arbitrary header:

```
curl -X POST https://target/perps/open_market_order \
 -H "Authorization: PublicKey 0xFAKE" \
 -d "name=BTC-USD&sz=0.1&is_buy=true"
```

The request is accepted and executed without ownership validation.

Location of Affected Code

File: perps/views.py

File: spot/views.py

Impact

Complete impersonation, any attacker can act as any wallet holder, performing trades or other privileged actions

Recommendations

Implement cryptographic challenge–response authentication: - Server issues a random nonce. - Client signs it with its private key. - Server verifies signature.

Alternatively, use OAuth2 or JWT authentication.

Team Response

Fixed.

[M-01] Debugging Imports and Pdb Usage

Severity

Medium Risk

Description

`pdb` is imported and used and there are many `print()` debug lines. Debugging hooks can hang processes (if `pdb.set_trace()` left) and `print()` leaks data to logs.

There are many print and potential stacktraces enabled that can uncover a surface for a potential attacker to enumerate or disclose information from the environment, the severity is stated accordingly to the sensitivity of the backend.

This refers to OWASP A09:2021 Security Logging and Monitoring Failures (logging secrets).

Vulnerable Scenario:

```
import pdb
...
print(coin_name)
print(order_result)
```

Location of Affected Code

`import pdb` excessive `print()`

Impact

Process availability issues, information leakage, and secret exposure in logs.

Recommendations

Remove pdb and debugging prints from production code. Use structured logging with redaction and conditional debug logging controlled by environment variables (and ensure DEBUG=False in prod).

Team Response

Fixed.

[M-02] Missing Input Validation and Unsafe Numeric Casting

Severity

Medium Risk

Description

User-provided parameters (oid, sz, px) are cast directly to integers/floats without validation or exception handling, allowing malformed or extreme input.

Unbounded, unvalidated input is a fundamental design flaw (A04). Attackers can flood endpoints with malformed payloads, causing high CPU usage or service instability.

This is referred to OWASP Mapping: A04:2021 – Insecure Design, API4:2023 – Unrestricted Resource Consumption

Vulnerable Scenario:

```
oid = int(request.POST.get("oid"))
sz = float(request.POST.get("sz"))
price = float(request.POST.get("px"))
```

Location of Affected Code

File: perps/views.py

File: spot/views.py

Impact

- Denial of Service (unhandled exceptions or heavy parsing).
- Invalid trades or corrupted state propagation.
- Information leakage via unhandled exceptions with DEBUG=True.

Recommendations

- Enforce schema validation (Django forms, DRF serializers, or Pydantic).
- Define numeric bounds and precision limits.
- Return HTTP 400 for invalid input.
- Implement request rate limiting and input sanitization.

Team Response

Fixed.

[M-03] Returning Raw Exceptions in API Responses

Severity

Medium Risk

Description

Many handlers return `JsonResponse({"error": str(e)})` on exceptions, exposing internal error messages and sometimes library paths or values.

Vulnerable Scenario: The following code is vulnerable

```
except Exception as e:
    return JsonResponse({"error": str(e)}, status=500)
```

Location of Affected Code

File: perps/views.py

File: spot/views.py

Impact

Reveals stack/context that helps attackers pivot or craft more effective payloads.

Recommendations

Return generic error messages; log details server-side only. Map exceptions safe error codes/messages.

Team Response

Fixed.

[M-04] Incorrect `update_leverage` Parameter Order Causes Wrong Leverage/Margin Mode

Severity

Medium Risk

Description

The function `exchange.update_leverage(...)` is invoked inconsistently: one call uses keywords (safe), while another uses positional arguments in the wrong order. The positional call passes (name, is_cross, leverage) clashes with the intended parameter mapping implied by the keyword call (name=..., leverage=..., is_cross=...). This likely swaps `leverage` and `is_cross`, resulting in incorrect leverage settings and margin mode.

Vulnerable Scenario:

```
# backend-main/perps/views.py
165:    lev_raw = request.POST.get("leverage")
166:    is_cross = request.POST.get("is_cross", "true").lower() == "true"
167:    if lev_raw is not None:
168:        leverage = int(lev_raw)
169:        # WRONG: positional order mismatched (name, is_cross, leverage
    )
170:        lev_result = exchange.update_leverage(name, is_cross, leverage
    )
171:        if not (isinstance(lev_result, dict) and lev_result.get("
    status") == "ok"):
172:            return JsonResponse({"error": f"Failed to set leverage: {
    lev_result}"}, status=400)
```

Non-Vulnerable Reference (keywords, safe)

```
# backend-main/perps/views.py
73:  lev_raw = request.POST.get("leverage")
74:  is_cross = request.POST.get("is_cross", "true").lower() == "true"
76:  leverage = int(lev_raw)
78:  lev_result = exchange.update_leverage(leverage=leverage, name=
    coin_name, is_cross=is_cross)
```

Location of Affected Code

File: perps/views.py#L74-L80

```
# --- Place market order ---
order_result = exchange.market_open(coin_name, is_buy, sz, None,slippage)
if order_result["status"] == "ok":
    for status in order_result["response"]["data"]["statuses"]:
        try:
            filled = status["filled"]
            return JsonResponse({"message": "Market Order placed", "
                result": filled})
```

File: perps/views.py#L165-L171

```
 # --- Order options ---
tif = request.POST.get("tif", "Gtc")  # Gtc (default), Ioc, or Alo
reduce_only = request.POST.get("reduce_only", "false").lower() == "true"

order_type = {"limit": {"tif": tif}}
if reduce_only:
    order_type["reduceOnly"] = True  # SDK respects this flag on
        order_type
```

Impact

Silent misconfiguration: If the SDK coerces types, the API might report success even though leverage/margin mode are wrong.

Recommendations

Fix the call at `perps/views.py:170` by using keywords (preferred, order-safe)

Team Response

Fixed.

[L-01] `slippage` Parameter Retrieved but Never Used

Severity

Low Risk

Description

The `slippage` parameter was retrieved but never used (lines 132, 170):

```python
slippage = float(slippage_raw) if slippage_raw is not None else None
```

The slippage is parsed from the request but never actually used in the order placement. The function `get_best_price()` hardcodes a 1% slippage instead.

Location of Affected Code

File: spot/views.py

Recommendations

Refactor the function to utilize the parameter.

Team Response

Fixed.

[L-02] No HTTPS / Secure Cookie Enforcement Settings in `settings.py`

Severity

Low Risk

Description

Secure cookie or HTTPS enforcement settings such as `SECURE_SSL_REDIRECT`, `SESSION_COOKIE_SECURE`, `CSRF_COOKIE_SECURE`, `SECURE_HSTS_SECONDS`, etc., were not found.

Location of Affected Code

File: rePRICE/settings.py

Recommendations

For production (example): - `SECURE_SSL_REDIRECT = True` - `SESSION_COOKIE_SECURE = True` - `CSRF_COOKIE_SECURE = True` - `SECURE_HSTS_SECONDS = 31536000` (and related HSTS settings) - Ensure TLS/HTTPS is enforced at the load balancer/web server.

Team Response

Acknowledged.

our shielding · Your smart contracts, our shielding · Your smart c

shieldify

Thank you!